

Fundação Getúlio Vargas
Estrutura de Dados

Relatório Lista 3

Alunos:

Pedro Marrochio Lacerda de Abreu - 251704025

Pedro Ricardo Alves Freitas — 251708013

Professor:

Matheus Telles Werner

17 de maio de 2026

Conteúdo

1	Algoritmos de ordenação	2
1.1	Insertion Sort	2
1.2	Merge Sort	2
1.3	Uso da ordenação	2
2	Análise de complexidade	2
3	Estrutura dos testes de desempenho	2
3.1	Gerador aleatório de jogadores	2
3.2	Geração de Matchmaking e execução de medições	2
4	Parâmetros utilizados	3
5	Resultados e comparações	3

1 Algoritmos de ordenação

1.1 Insertion Sort

O algoritmo de ordenação por *InsertionSort* foi implementado diretamente dentro da classe **Matchmaking**, ordenando o próprio array **players**. A lógica utilizada foi ordenar os jogadores por score, realizando também verificação do *timestamp* unicamente para garantir a ordenação (mesmo o algoritmo sendo estável).

1.2 Merge Sort

O algoritmo de ordenação por *MergeSort* necessitou de uma adaptação para ser implementado dentro da classe **Matchmaking**, já que o fato da função **sortByScoreMerge** não possuir retorno dificultou a implementação da recursão. Para isso, no arquivo **AuxiliarFunctions.cpp** foi implementada a lógica de ordenação em duas funções, **merge_list** e **merge_sort**. A primeira é responsável pela concatenação de duas listas e a segunda é responsável pela lógica de recursão do algoritmo. Assim, essas funções auxiliares foram utilizadas dentro da classe **Matchmaking** para ordenar diretamente o array **players**.

1.3 Uso da ordenação

Além disso, como o array de players já estava ordenado, bastou que a procura do grupo fosse feita analisando o jogador na primeira posição e aquele localizado $k - 1$ posições à frente dele no array **players**, em que k é a quantidade desejada de jogadores do grupo. Caso essa primeira análise não encontrasse um grupo válido, a lógica seguiria, mas agora incrementando em 1 a posição do primeiro jogador e do $k - 1$ posições a frente dele, para agora checar o grupo entre os jogadores 2 até k e assim sucessivamente. Essa lógica é assegurada pela ordenação, já que todos os jogadores entre o primeiro e o último do grupo têm diferença menor em relação ao jogador com score mais alto, nesse caso o último, ou seja, a diferença entre o último e o primeiro jogador serve como teto do grupo e, portanto, é o único valor necessário para comparação com o delta desejado. Além disso, a remoção também se torna mais simples, já que todos os jogadores entre o primeiro e o último do grupo escolhido fazem parte dele, tendo uma faixa clara de remoção.

2 Análise de complexidade

Inserção: Como todos os jogadores são inseridos no final do array **players** e temos a informação de seu tamanho, conseguimos um acesso direto da memória, fazendo com que a complexidade da inserção seja $O(1)$.

Remoção: Existem dois casos quanto à remoção. No primeiro, o jogador existe no array e temos que percorrê-lo, jogando todos os jogadores a partir do jogador encontrado uma posição para trás. No segundo, o jogador não existe e tivemos que percorrer todo o array para verificar essa informação. Assim, no pior caso, a remoção tem complexidade $O(n)$.

Ordenação com Insertion: Como para cada jogador a ser ordenado percorremos todo o array, temos que a complexidade desse algoritmo é $O(n \times n) = O(n^2)$.

Ordenação com Merge: Vamos analisar o algoritmo por níveis. No primeiro nível, temos um problema com complexidade cn para ser resolvido. No segundo nível temos dois problemas com complexidade total $2 \times c \times \frac{n}{2} = cn$. Assim, todos os níveis de recursão têm complexidade total de cn para serem resolvidos. Sabemos que a divisão de níveis para quando $\frac{n}{2^k} = 1$, ou seja, existem $k = \log(n)$ níveis. Assim, a complexidade total é $O(cn \times \log(n)) = O(n \times \log(n))$.

Formação do grupo: Como existe a procura do grupo e sua remoção, caso encontrado, a complexidade desse método é análoga ao método de remoção, sendo ela $O(n)$.

Obtenção de jogadores: Como o método **getWaitingPlayers** apenas retorna elementos, e o acesso a esses é imediato, então sua complexidade é $O(1)$.

3 Estrutura dos testes de desempenho

3.1 Gerador aleatório de jogadores

A necessidade de geração de vários jogadores tornou necessária a automatização desse processo. Para isso, foram criadas as funções **score_aleatorio** e **gerar_player_aleatorio**. A primeira gera um número entre 0 e 1000 para servir como score de um player qualquer. A segunda utiliza de um *id* e um *timestamp* dados, além da função **score_aleatorio** para retornar um jogador com score gerado aleatoriamente.

3.2 Geração de Matchmaking e execução de medições

Para os testes, foi estruturada a seguinte lógica dentro da função **main**: Primeiro, foram definidos o número de jogadores utilizados em cada registro de ordenação e o número de repetições que esse registro iria ocorrer. Assim, foram realizados loops que criam

dois *Matchmaking*, um para ser ordenado pelo *InsertionSort* e outro pelo *MergeSort*. Então, é criado um array de *Player* com *score* aleatórios dentro de cada *Matchmaking*, para serem ordenados um determinado número de vezes e terem seus tempos registrados por funções da biblioteca *chrono*. Então, a média desses tempos é obtida e registrada em um array, o qual é impresso para que seja possível a visualização.

4 Parâmetros utilizados

Para os testes, foi registrada a média de tempo de 5 repetições de cada método de ordenação, com uma lista de 5000, 10000, ..., 50000 players. A quantidade de jogadores foi escolhida com o objetivo de atingir um número grande de elementos, passando por intervalos de interesse igualmente espaçados.

5 Resultados e comparações

Veja abaixo a tabela e os gráficos com os resultados obtidos das medições:

Tabela 1: Tempo de execução dos algoritmos em microsegundos

Quantidade de jogadores	InsertionSort	MergeSort
5000	91576	3719
10000	372462	8310
15000	815578	11587
20000	1441479	17138
25000	2247619	20836
30000	3281211	28246
35000	4401846	30901
40000	5741816	38370
45000	7189251	40988
50000	9065767	44463

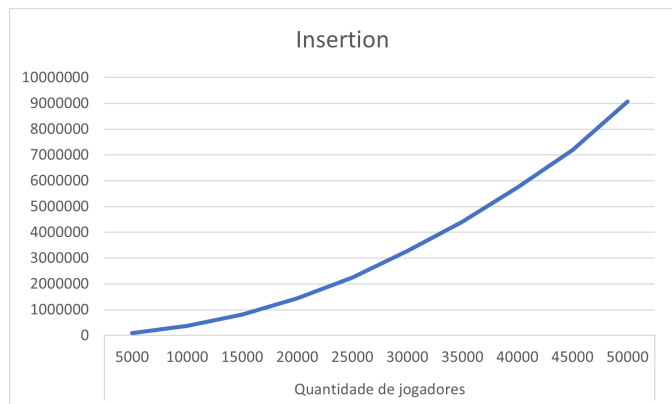


Figura 1: Medição InsertionSort

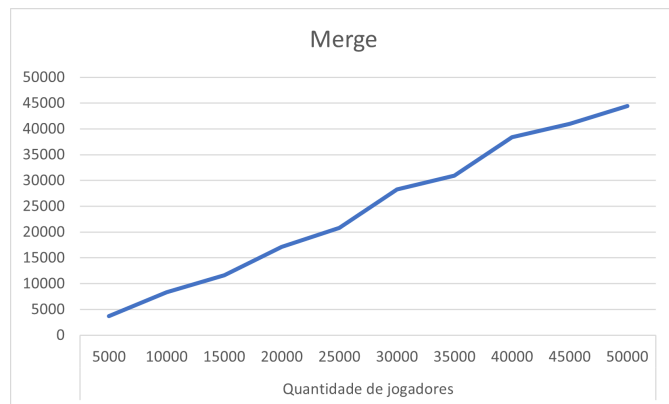


Figura 2: Medição MergeSort

Analisando os valores da Tabela 1, podemos perceber a diferença na grandeza dos tempos medidos entre os dois algoritmos (mesmo esse tempo estando em uma unidade muito pequena). Além disso, é possível ver também que os tempos registrados no *InsertionSort* crescem em maior velocidade em comparação aos tempos registrados no *MergeSort*.

Já o comportamento da complexidade dos algoritmos pode ser melhor observado nos gráficos acima, produzidos no Excel com os dados da Tabela 1. A Figura 1 representa os tempos medidos com *InsertionSort*, e é possível observar que o crescimento segue o comportamento da complexidade desse algoritmo, que é $O(n^2)$. Já na Figura 2, podemos ver a representação dos tempos medidos com *MergeSort*, observando um comportamento quase linear (com uma pequena variação entre 25000 e 40000). Isso se deve à complexidade do algoritmo, $O(n \times \log(n))$, que tem um fator linear predominante, fazendo com que o gráfico se assemelhe a uma reta.